

MuKV: Multi-Grained KV Cache Compression for Long Streaming Video Question-Answering

Junbin Xiao^{1,2*}, Jiajun Chen^{2*}, Tianxiang Sun², Xun Yang¹, Angela Yao²
¹ University of Science and Technology of China, ² National University of Singapore
 junbinxiao@ustc.edu.cn, chen.jiajun@u.nus.edu, angela.yao@nus.edu.sg

Abstract

Long streaming video QA remains challenging due to growing visual tokens and limited reasoning length of large language models (LLMs). KV-caching stores the Key-Value (KV) of the historical tokens via LLM prefill and enables more efficient streaming QA. However, existing methods cache every one or two frames, causing redundant memory usage and losing fine-grained spatial details within frame or temporal contexts across frames. This paper proposes MuKV, a method that features a multi-grained KV cache compression module and a semi-hierarchical retrieval approach to improve both efficiency and accuracy for long streaming VideoQA. For the offline KV cache, MuKV extracts visual representations at patch-, frame-, and segment-levels. The multiple levels of granularity preserve both local cues and global temporal context, while maintaining efficiency with a dual signal token compression mechanism guided by self-attention and frequency. For online QA, MuKV designs a semi-hierarchical retrieval method to retrieve relevant KV caches for answer generation. Experiments on long-streaming VideoQA benchmarks show that MuKV significantly improves answer accuracy, without sacrificing memory and online QA efficiency. Moreover, our compression mechanism alone brings consistent benefits across answer accuracy, memory, and QA efficiency over baselines, showcasing highly effective contribution.

1. Introduction

Multimodal large language models (MLLMs) have enabled remarkable progress in VideoQA [1, 3, 21, 22, 26, 35, 38, 51, 53]. Yet, most of the advances are made on understanding relatively short or offline videos with determinate lengths [12, 42, 44, 49]. Extending such capabilities to online streaming videos of unconstrained length remains a significant challenge. The core difficulty lies in the linear growth of visual tokens with time, which quickly exceeds

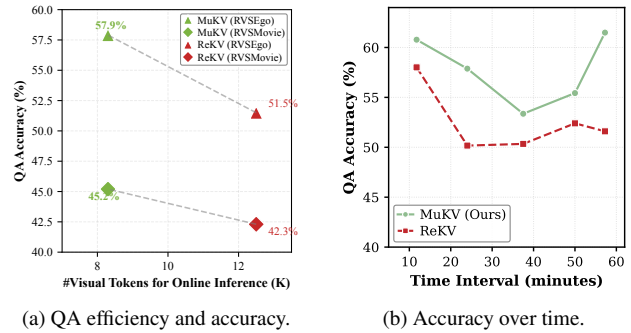


Figure 1. Comparison between MuKV and previous arts. (a) MuKV improves ReKV’s answer accuracy, without increasing on-line QA time and offline KV storage. (b) The advantage gets strengthened over time in streaming video QA.

the reasoning context window of LLMs, and leads to inefficiency and reduced answer accuracy.

To address this challenge, primary approaches focus on extending LLM context window [32, 37, 40, 45, 47, 55], e.g., via token compression, though this may require large-scale pretraining, incur quadratic computation costs, and trade off visual details for long-ranged modeling. As such, this strategy is limited in its applicability for streaming QA. A second approach uses decoupled *offline memory* and *online retrieval* mechanisms [4, 9, 11, 19, 30, 31, 36], with the LLM context window unaltered. They offline store either visual descriptions [4, 31, 39, 52], embeddings [4, 31, 54], or Key-Value (KV) caches [9, 19] of the past video streams, and online retrieve a subset of relevant information for answering when a question triggered. Among them, KV-caching enables training-free and efficient online answer decoding (without redundant KV computation of history tokens), showing promising streaming QA applicability.

However, existing KV-cache approaches [9, 19] are limited to per-frame caching, the frame-level representations alone may hardly encode region-level visual details and cross-frame temporal contexts. Also, the linearly increased KV cache brings significant redundancy for storage, which may affect retrieval and consequently QA performance,

*Equal Contribution.

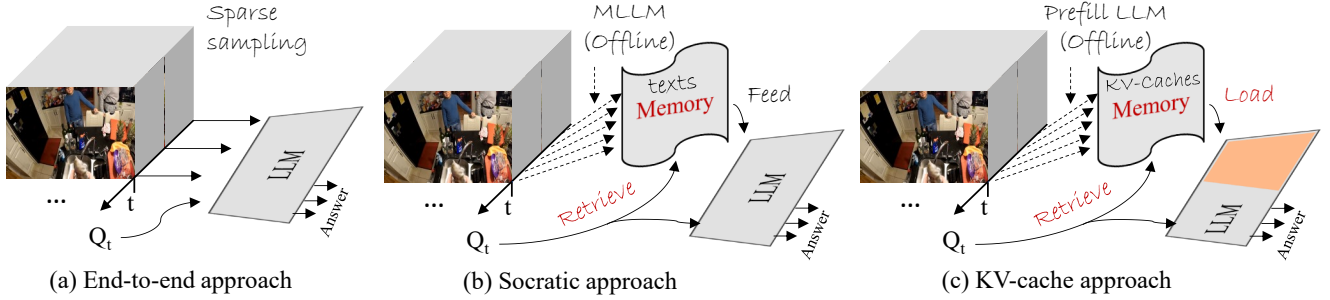


Figure 2. Illustration of different approaches for streaming video QA. (a) The end-to-end approach trades off visual details for long-ranged modeling. (b) The Socratic approach suffers from online efficiency since it consumes full LLM computation online. (c) The KV-cache approach enables more efficient online QA via exemption of online KV-computation of past visual tokens.

leaving open challenges for effective KV-cache memory, compression, and retrieval [6, 19, 29, 46].

In this paper, to improve KV-cache memory and retrieval, we propose MuKV, a novel method that underscores a multi-grained KV-cache compression module and a semi-hierarchical retrieval mechanism for more effective long streaming VideoQA. For offline memory, MuKV extracts video KV caches at multiple granularity levels: patch, frame, and segment — to preserve spatial details and maintain temporal contexts. Importantly, to reduce the cache for efficient memory and retrieval, MuKV proposes a Dual signal KV-cache Compression (DCP) mechanism guided by self-attention importance and token frequency to prune redundant KV caches. For online QA, MuKV designs a semi-hierarchical retrieval strategy that first retrieves in parallel among KV caches across all three granularities with the question as query, and then reranks the top-ranked KV tokens via coarse-to-fine cross-granularity retrieval. The final top-ranked KV caches are loaded into LLMs along with the question for answer generation. Extensive experiments demonstrate that MuKV achieves significant accuracy improvements in QA over long video streams, without sacrificing memory and online QA efficiency (Fig. 1).

To summarize our contributions:

- We propose MuKV, a streaming VideoQA approach that highlights a novel offline multi-grained KV-cache compression module and an online semi-hierarchical retrieval strategy to improve answer accuracy, yet without sacrificing memory and online QA efficiency.
- We propose DCP compression mechanism, and demonstrate that frequency signals along with self-attentions are effective indicators for video KV-cache compression.
- Our method shows stronger advantage as the video stream continues to grow longer.

2. Related Work

2.1. MLLMs for Long Streaming VideoQA

Large language models (LLMs) [7, 10, 15, 47] has stimulated a surge of multimodal LLMs (MLLMs) that enable

freely chat with images and videos [3, 20, 22, 26, 35, 38, 43, 53]. Despite the success, the end-to-end MLLMs (Fig. 2(a)) often require large-scale pretraining and trade off visual details for long video modeling [17, 32, 40, 54, 55], with also severely reduced efficiency for the quadratic increase of computation, making them less-practical in coping with dynamic video streams.

In contrast, Socratic or agentic approaches [11, 28, 36, 50, 52] leverage pretrained MLLMs for long streaming VideoQA by introducing offline memory and online retrieval modules (Fig. 2(b)) to store and recall past visual information (*e.g.*, visual descriptions or embeddings). While effective, both descriptions and embeddings demand full online LLM computation, leading to high inference latency. In addition, descriptions alone may not be accurate and sufficient to answer questions in streaming QA practice. The recent ReKV [9] introduces storing the KV cache as a compromise between visual embeddings and language descriptions (Fig. 2(c)). However, per-frame KV caches are insufficient for video modeling (*e.g.*, lacking spatial details and temporal contexts) whilst consuming large amounts of storage that in turn reduces retrieval and QA performance.

Our approach, MuKV, follows the fashion of offline KV-cache memory and online retrieval to achieve streaming QA (Fig. 2(c)). In contrast to ReKV [9], which indiscriminantly stores per-frame KV caches without considering information granularity and redundancy, we introduce to compactly represent the video at multiple granularities on a per-segment basis achieved by a novel KV-cache compression mechanism. Such designs allow MuKV to retain better the spatial/temporal fidelity of videos for improved QA, yet achieve sub-linearly increased memory to account for continuous video streamings.

2.2. KV-Cache Compression

To compress the KV-cache, existing algorithms focus on token quantization [14, 25] and pruning [16, 33, 41]. Quantization may introduce approximation errors and hurt prediction accuracy. Thus, we focus on pruning to reduce redundancy. The popular indicators for token pruning are

self-attention score [13, 16, 19, 48] and token similarity [5, 19]. While self-attention scores are easy to obtain (during LLM prefilling), token similarity often introduces non-trivial computation costs because of pairwise token comparison. Recent work InfiniPot [19] also introduces Value Norm as an effective and more efficient indicator. However, it limits in selecting salient frame regions in spatial domain, failing to analyze across temporal frames and segments.

In MuKV, we introduce an equally efficient but more generalized indicator – frequency, with the intuition that token frequency distribution within a frame and across frames helps localize semantically important regions and moments respectively. The closest work FreqKV [18] also uses frequency as an indicator for KV-cache compression, though they target reducing text tokens of high frequency for online natural language decoding. MuKV differs fundamentally by focusing on video KV cache compression, also at multi-granularity levels of spatial patches, temporal frames, and segments, to reduce offline memory redundancy, in which we find that tokens of high frequency to be more important.

2.3. KV-Cache Retrieval

KV-cache retrieval retrieves a subset of token KV values in the cache relevant to the query for decoding the answer. Existing progresses on pure text QA [24, 34] directly exploit self-attention scores. For cross-modal retrieval in video QA, pioneer approaches [9, 19, 29] consider cross-modal cosine similarity. However, they apply parallel retrieval among all tokens, which may bring noises when coping with multi-grained information. While hierarchical retrieval [39] shows benefits, it suffers from error propagation if the top-level retrieval is wrong. For balance, we design a semi-hierarchical retrieval method by firstly performing hierarchy-agnostic parallel retrieval and then reranking the retrieved visual tokens at lower granularity via cross-grain hierarchical retrieval. Our experiments show that this semi-hierarchical approach effectively reduces noises and improves answer accuracy, despite slightly reducing efficiency due to reranking.

3. Method

3.1. Problem and Method Overview

Problem Formulation. Given a streaming video $\mathcal{V} = \{f_t\}_{t=1}^T$ arriving sequentially and a series of user questions $\mathcal{Q} = \{q_i\}_{i=1}^M$ triggered at different timestamps $\{t_i\}_{i=1}^M$, streaming VideoQA aims to generate answers $\mathcal{A} = \{a_i\}_{i=1}^M$ according to the video contents up to t_i for each question. Since the user questions are not known in advance, effective streaming video QA system must: (i) maximally maintain critical past visual information for answering unknown questions, (ii) within affordable memory budgets, and (iii) enable real-time online QA.

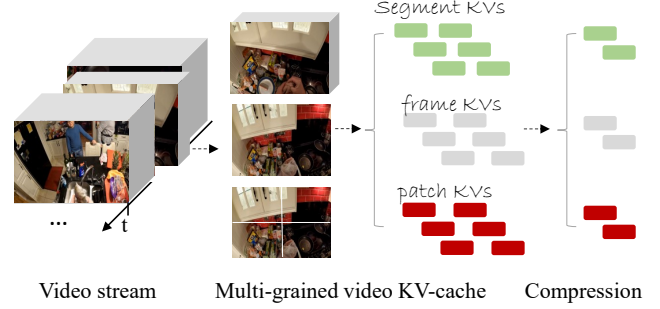


Figure 3. Illustration of multi-grained video KV cache compression in offline memory.

Method Overview. To achieve streaming VideoQA, we propose MuKV which follows the KV-cache framework shown in Fig. 2(c). It mainly includes two modules: offline KV-cache memory to memorize KV caches of past video streams and online KV-cache retrieval to retrieve question relevant video KV caches for online answer generation. On top of this framework, we design (1) **a multi-grain KV cache compression** module that compactly stores the past video KV-caches of multi-granularities offline, and (2) **a semi-hierarchical KV cache retrieval** mechanism for efficient and accurate online QA. MuKV is training-free, model-agnostic, and can be integrated seamlessly with any existing Video-LLMs in principle. We describe in detail our two innovative designs in the next subsections.

3.2. Multi-Grained Video KV Cache

While previous methods [9, 19] treat a video stream as a sequence of frames. We encode a video stream $\mathcal{V}^T = \{v_1, v_2, \dots, v_T\}$ incrementally at a per-segment basis to embrace video information of multiple granularities within each segment. Specifically, as shown in Fig. 3, at each time step t , we derive three video representations from the current segment v_t : the whole segment v_t , the middle frame f_t of this segment, and the patches of the middle frame p_{f_t} . In practice, since each visual token in existing MLLMs (e.g., LLaVA-OV [20]) corresponds to an original frame patch, the three-level of representations can be derived by grouping visual tokens at different scales without multiple times of visual encoder inference. For example, assume there are F frames in each segment and P total patches in each frame, and token x_i represents a single patch, if we partition a frame into S super patches ($S < P$), then $v_t = \{x_i\}_{i=1}^O$ ($O = P \times F$), $f_t = \{x_i\}_{i=1}^P$, and $p_{f_t} = \{x_i\}_{i=1}^{\lfloor P/S \rfloor}$ can effectively represent a segment, a middle frame of the segment, and a super patch of the middle frame, respectively.

To obtain the KV caches for the current segment, the above token representations at each granularity level (e.g., f_t), along with all the past KV caches at the same gran-

ularity level (e.g., $\{(\mathbf{K}_{f_{1:t-1}}^{(\ell)}, \mathbf{V}_{f_{1:t-1}}^{(\ell)})\}_{\ell=1}^L$), are independently fed into LLM to obtain the KV caches at this time step (e.g., $\{(\mathbf{K}_{f_t}^{(\ell)}, \mathbf{V}_{f_t}^{(\ell)})\}_{\ell=1}^L$), where L is the number of transformer layers. At the same time, we obtain attention weights $\{\mathbf{A}_{f_t}^{(\ell)}\}_{\ell=1}^L$ as indicators for subsequent KV cache compression. Note that each time step performs three LLM prefills to obtain the three granularity KV caches. Moreover, the three prefills are executed in parallel in practice and incur no additional latency.

3.3. Dual-Signal KV-Cache Compression

Multi-grained video KV caches will result in heavy memory redundancy. For compression, we design a dual-signal KV-cache compression module (DCP) by jointly considering self-attention score and token frequency. Since our compression is independently applied for KV caches of different granularities. We use the frame-level representations as an example to introduce the details below.

Attention-based Indicator. Self-attention score serves as an ideal indicator for token selection, since it naturally reflect a token’s importance among the token sequence and is readily available along with KV caches without additional computation. In our implementation, the attention scores of the last LLM layer are chosen as effective indicator, as we empirically find that there is no obvious distribution pattern among the attention scores of the intermediate layers. Concretely, the token importance scores within f_t is obtained by aggregating the last-layer attention across multiple heads of all tokens:

$$\mathbf{I}_{\text{att}} = \frac{1}{H \cdot P} \sum_{h=1}^H \sum_{i=1}^P \mathbf{A}_{h,i}^{(L)}, \quad \mathbf{I}_{\text{att}} \in \mathbb{R}^{P \times 1}, \quad (1)$$

where H denotes the number of self-attention heads.

Frequency-based Indicator. While attention score reflects a token’s semantic importance to the pretrained task (e.g., captioning or question answering), it may overfit and generalize poorly to new tasks. Thus, we consider capturing task-agnostic video characteristics as indicators for removing visual redundancy. To this end, we leverage the Fast Fourier Transform (FFT) algorithm [8] to transform token key vectors to the frequency domain. The core intuition is that token frequency signals content variability – static or redundant content often shows lower frequency compared to changing and dynamic content. Furthermore, frequency can also be efficiently calculated using FFT.

Specifically, to derive a frequency score that correlates with each token, we first apply FFT along each token dimension across a sequence of key token representations $\{\mathbf{k}_i\}_{i=1}^P$.

$$\mathbf{Z}_{\text{fft}} = \text{FFT}(\mathbf{k}^{P \times D}), \quad \mathbf{Z}_{\text{fft}} \in \mathbb{R}^{P \times D}. \quad (2)$$

Then, the frequency scores $\mathbf{I}_{\text{fft}}$ are obtained by averaging the magnitudes across all dimensions of each token’s frequency

representation $\mathbf{Z}_{\text{fft}}$:

$$\mathbf{I}_{\text{fft}} = \text{Mean}(\mathbf{Z}_{\text{fft}}^{P \times D}), \quad \mathbf{I}_{\text{fft}} \in \mathbb{R}^{P \times 1}. \quad (3)$$

We perform simple mean-pooling along dimensions because token of higher frequency often shows larger magnitude (and lower frequency smaller ones) in most dimensions of its frequency representation.

Dual Signal Fusion. Finally, we fuse the attention and frequency scores via a weighted sum:

$$\mathbf{I}_{f_t} = \alpha_{f_t} \hat{\mathbf{I}}_{\text{att}} + (1 - \alpha_{f_t}) \hat{\mathbf{I}}_{\text{fft}}, \quad \mathbf{I}_{f_t} \in \mathbb{R}^{P \times 1}, \quad (4)$$

where $\hat{\mathbf{I}}$ denotes min-max normalization to $[0, 1]$, and α_{f_t} controls the attention importance and frequency score trade-off. Similarly, we can obtain indicator scores $\mathbf{I}_{p_{f_t}} \in \mathbb{R}^{[P/S] \times 1}$, $\mathbf{I}_{v_t} \in \mathbb{R}^{O \times 1}$ for KV caches at the patch and segment levels, respectively.

Granularity Adaptive Compression. According to the fused importance scores $\mathbf{I}_{v_t}, \mathbf{I}_{f_t}, \mathbf{I}_{p_{f_t}}$, we sort in descending order and keep the top $\kappa_{v_t} = \lfloor \rho_{v_t} \cdot |v_t| \rfloor$, $\kappa_{f_t} = \lfloor \rho_{f_t} \cdot |f_t| \rfloor$, $\kappa_{p_{f_t}} = \lfloor \rho_{p_{f_t}} \cdot |p_{f_t}| \rfloor$ tokens, where ρ_{v_t} , ρ_{f_t} , and $\rho_{p_{f_t}}$ are granularity-specific hyper-parameters. Our motivation is that different granularities serve different functional roles: segment provides semantic narrative contexts, while patch captures regional changes. Thus, we apply different compression ratio for reducing tokens at different granularities. Finally, the selected KV slices $\{(\mathbf{K}_p^{(\ell)}, \mathbf{V}_p^{(\ell)})\}_{\ell=1}^L$, $\{(\mathbf{K}_f^{(\ell)}, \mathbf{V}_f^{(\ell)})\}_{\ell=1}^L$, $\{(\mathbf{K}_v^{(\ell)}, \mathbf{V}_v^{(\ell)})\}_{\ell=1}^L$ of different granularities are stored with their corresponding temporal timespans, forming a multi-grained yet compact KV cache.

3.4. Semi-Hierarchical KV-Cache Retrieval

For online QA, we utilize a two-stage semi-hierarchical retrieval mechanism to obtain a subset of query-relevant video KV caches. As shown in Fig. 4, we first retrieve in parallel among KV caches across all three granularities with the question as query, and then rerank the top-ranked KV caches via coarse-to-fine cross-granularity hierarchical retrieval. The final subset of top-ranked KV caches are loaded into LLM for answer decoding.

Specifically, within each granularity KV cache block (A block can be understood as a frame, a segment, or a patch, depending on what information granularity being processed. We call block as they are incompleter representations because of token compression or pruning), e.g., $\{(\mathbf{K}_{f_t}^{(\ell)}, \mathbf{V}_{f_t}^{(\ell)})\}_{\ell=1}^L$, we mean-pool the last-layer key vectors across the block (multi-heads are concatenated), e.g.,

$$\mathbf{k}_{f_t} = \frac{1}{N_p} \sum_{j=1}^{N_p} \mathbf{k}_j, \quad \mathbf{k}_{f_t} \in \mathbb{R}^C, \quad (5)$$

where $N_p < P$ denotes number of reserved tokens in the frame-level block, and $C = H \times D$ denotes feature dimension. Similarly, we can obtain $\mathbf{k}_{v_t}$ and $\mathbf{k}_{p_{f_t}}$.

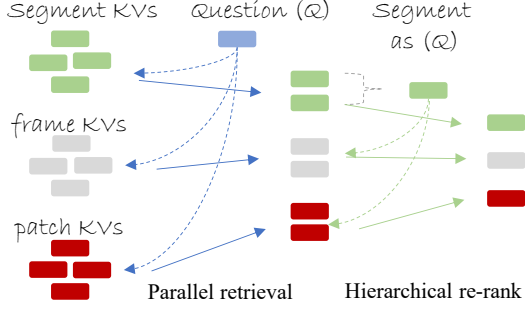


Figure 4. Illustration of semi-hierarchical retrieval.

Stage-1: Parallel Retrieval. For parallel retrieval at the first stage, we mean-pool the last-layer question query tokens (multiple heads are concatenated) to obtain the global question representation:

$$\mathbf{q} = \frac{1}{N_q} \sum_{k=1}^{N_q} \mathbf{q}_k, \quad \mathbf{q} \in \mathbb{R}^C, \quad (6)$$

where N_q is the number of tokens in the question. At each granularity, we compute the cosine similarity between the question query $\mathbf{q}$ and all video key representations prior to the question-triggered timestamp t (e.g., $\mathbf{K}_{f_t}^{t \times C}$):

$$s_{f_t} = \cos(\mathbf{q}, \mathbf{K}_{f_t}), \quad \mathbf{K}_{f_t} \in \mathbb{R}^{t \times C}. \quad (7)$$

Then, we retrieve top- $2k_g$ blocks per granularity according to s , where k_g is granularity-specific hyper-parameters.

Stage-2: Hierarchical Reranking. The above grain-agnostic parallel retrieval will introduce noises as the model may show undesirably strong cross-model responses on some local visual contents. Thus, to enforce global coherence, we further apply the video representations at high granularity to identity the noises at lower granularities via cross-grained hierarchical retrieval. As shown in Fig. 4 (right), to accomplish this, we first obtain a global query by averaging the top- N segment-level representations retrieved at the first stage. For block candidate j at lower granularities, we compute a consistency score γ_j , measuring its alignment with this global query, and update its ranking score via:

$$\tilde{s}_j = (1 - \lambda_g) s_j + \lambda_g \gamma_j, \quad (8)$$

where λ_g is a coherence enforcement factor. Blocks at lower granularity require higher λ_g to anchor them within the global segment, while for segment-level blocks, we set $\lambda_g = 0$ since the global queries are derived from them. Finally, we select the top- k_g blocks per granularity based on this calibrated score $\tilde{s}$.

3.5. Question-Answering Using Retrieved KV.

The retrieved video KV caches serve as the context for Video-LLM question-answering. Formally, the attended

values are calculated as:

$$a_t = \text{LLM}(\mathbf{W}_Q \mathbf{X}, [\mathbf{R}_k, \mathbf{W}_K \mathbf{X}], [\mathbf{R}_v, \mathbf{W}_V \mathbf{X}]), \quad (9)$$

where $\mathbf{X}$ represents either the question tokens or the current token being decoded, and $\mathbf{R}_k$ and $\mathbf{R}_v$ are the key and value vectors from the context, which include both retrieved video KV and previously generated tokens. The LLM generates answer a_t autoregressively. For subsequent queries, the process repeats with updated timestamp constraints (e.g., $t = t + 1$), allowing multiple questions to be answered efficiently in a streaming manner.

4. Experiment

4.1. Experimental Setup

Datasets and Evaluation. We evaluate MuKV on three long streaming video QA datasets: RVSEgo [54], RVS-Movie [54], and StreamingBench [23]. Extended experiments on offline long video QA datasets [12, 27, 56] are presented in the Supplementary. RVSEgo includes 1.4k questions over 11 ego view videos of averaging 30 minutes. RVS-Movie includes 1.9k questions over 20 movie videos of averaging 1 hour. StreamingBench includes 4.5K questions over 900 videos, in which we focus on the subset which emphasizes real-time visual understanding, including 2.5k questions over 500 videos of averaging 10 minutes. Other dataset details are presented in the Supplementary.

We compare with previous arts under three metrics: online QA accuracy, efficiency, and offline memory (cache size). For effective accuracy comparison on RVSEgo and RVS-Movie, we directly adopt the evaluation script provided by [9, 54] which uses LLM as answer judge. Unfortunately, the default LLM judge GPT-3.5-turbo-0613 has been deprecated, so we use GPT-3.5-turbo. This results in slight accuracy differences (specified in Tab. 1). On StreamingBench, we report standard multi-choice accuracy. For memory and inference efficiency, we report the number of cached or inference visual tokens as a device-independent efficiency metric, i.e., less tokens bring higher efficiency.

Configurations. We utilize the classical backbone model LLaVA-OV [20] (both 0.5B and 7B) to instantiate our MuKV framework. Our major experiments are conducted on A5000 GPUs with each 24G memory. We list the values of some key hyper-parameters, which are greedily searched on RVSEgo and applied to the other two datasets. Specifically, for each video, we sample at 0.5FPS unless otherwise indicated, and 4 continuous frames (spanning 8 seconds) constitute a segment, where a frame is divided into $S = 4$ super-patches (original patches $P = 196$) for multi-grained video representations. For retrieval, we retrieve a fixed number of $k_g = 64$ blocks following ReKV [9]. Other granularity specific hyper-parameters are: $\alpha = \{0.5, 0.7, 0.8\}$, $\rho = \{0.1, 0.1, 0.8\}$, $k = \{20, 32, 12\}$, and

Table 1. Streaming VideoQA performance comparison. For evaluation on VStream-QA [54], we use GPT-3.5-turbo, and gray out the previous results evaluated using GPT-3.5-turbo-0613 as it was deprecated. Less inference visual tokens and memory tokens indicate higher efficiency. For the number of memory tokens, we report on a per 300-frame (10 minutes) basis, *e.g.*, $59K \approx 196 * 300$.

Model	Size	#Inf. Tok ↓	# Mem. Tok ↓	VStream-QA		StreamingBench (Real-Time)											
				RVSEgo	RVSMovie	OP	CR	CS	ATP	EU	TR	PR	SU	ACP	CT	All	
FVStream [54]	7B	–	–	59.0	56.1	–	–	–	–	–	–	–	–	–	–	–	
ReKV [9]	0.5B	12.5K	59K	54.7	44.6	–	–	–	–	–	–	–	–	–	–	–	
ReKV [9]	7B	12.5K	59K	63.7	54.4	–	–	–	–	–	–	–	–	–	–	–	
LongVA (128F) [20]	7B	18.4K	0	–	–	70.0	63.3	61.2	70.9	62.7	59.5	61.1	53.7	54.7	34.7	60.0	
FVStream [54]	7B	–	–	58.5	54.7	27.1	45.4	25.9	25.0	28.5	13.6	19.4	26.3	24.9	50.2	24.3	
ReKV [9]	0.5B	12.5K	59K	51.5	42.3	55.3	57.0	61.2	60.5	56.0	50.1	48.1	46.3	51.1	33.0	52.7	
ReKV [9]	7B	12.5K	59K	56.2	48.2	71.3	70.3	69.1	68.6	61.6	55.2	63.9	58.1	55.7	46.3	62.3	
MuKV (Ours)	0.5B	8.3K	59K	57.9	45.2	60.7	67.2	65.0	63.5	61.0	51.7	63.0	50.1	54.0	32.4	56.8	
MuKV (Ours)	7B	8.3K	59K	59.5	48.5	74.0	78.2	72.2	71.8	64.8	55.5	66.7	58.9	59.4	39.4	64.4	

$\lambda = \{0.3, 0.3, 0\}$ for patch-, frame-, and segment-level information, respectively.

4.2. Main Results

Tab. 1 compares MuKV with existing long and streaming video QA methods. The results show that MuKV consistently outperforms other recent competitors on RVSEgo and StreamingBench. While FVStream [54] wins on RVSMovie, it generalizes extremely poor to StreamingBench. We speculate that FVStream likely overfits the training data since it is additionally learned instead of using existing MLLMs. Notably, MuKV achieves stable accuracy improvements over ReKV [9], without increasing the memory usage and even slightly boosting inference efficiency, demonstrating the high effectiveness and robustness of our innovation designs. In the Supplementary, we further show that MuKV’s performance can be significantly boosted. For example, the overall accuracy on StreamingBench improves to 71.4% when we increase the frame rate from 0.5FPS to 3FPS, suggesting its potential for long streaming VideoQA with a large memory budget. Moreover, by replacing the backbone model LLaVA-OV [20] with Qwen3-VL [2], MuKV can gain further improvements on popular long video QA tasks.

On StreamingBench, MuKV achieves remarkable performance improvements over LongVA [55] and ReKV [9] on questions about higher-level video understanding, such as “Causal Reasoning (CR)”, “Clip Summarize (CS)”, “Event Understanding (EU)”, and “Prospective Reasoning”, but fall short in answering questions of “Counting (CT)”. We speculate that our KV cache compression can effectively maintain the global semantics of a video segment but may be ill-suited for counting which is sensitive to fine-grained information changes.

Finally, the result visualization in Fig. 6 demonstrates MuKV’s superior behavior in reasoning both segment-level temporal dynamic and region-level details.

Table 2. Investigation of MuKV (0.5B) under different granularities. (All variants are under parallel retrieval and are with the same QA efficiency for fair accuracy comparison.)

Model	Pat.	Fra.	Seg.	# Inf. Tok ↓	# Mem. Tok ↓	Acc@Ego	Acc@Movie
MuKV	✓	✗	✗	8.3k	47k	51.6	44.1
MuKV	✗	✓	✗	8.3k	39k	53.1	45.2
MuKV	✗	✗	✓	8.3k	10k	54.9	44.8
MuKV	✓	✓	✗	8.3k	53k	55.1	43.6
MuKV	✓	✗	✓	8.3k	53k	55.2	45.1
MuKV	✗	✓	✓	8.3k	53k	55.4	43.7
MuKV	✓	✓	✓	8.3k	59K	56.5	46.0

4.3. Ablation Studies

Multi-granularity. Tab. 2 analyzes the impact of video KV representations at different granularities. Results in the top block show that segment-level KV representations alone generally outperform the other two granularities, suggesting that higher-level and temporally contextual semantics (*e.g.*, actions and events) play a more crucial role in long video understanding. In the middle block, combining segment-level representations with either frame- or patch-level ones consistently yields better results than other combinations and also surpasses those of using a single granularity. Finally, the bottom block shows that integrating all three granularities achieves the best performance, demonstrating the superiority of our multi-grained video KV representations. Notably, our accuracy gains incur no additional memory or online QA overhead owing to the compression mechanism.

KV Cache Compression. We experiment with different models to validate the effectiveness of our DCP compression in Tab. 3. Results in the top block show that DCP effectively reduces MuKV’s memory, and in turn improves QA efficiency and accuracy. Also, both self-attention scores and frequency scores serve as effective indicators for token importance. Notably, when retaining the same KV cache size with ReKV, *e.g.*, compressing the KV cache by 2/3 (67%), we earn significant performance boost over ReKV,

Table 3. Investigation of KV cache compression. Half frame: we adopt a more sparse sampling rate of 0.25 FPS. Rand(50%): we randomly drop half of tokens. DCP(ratio): we drop {ratio} of tokens using our DCP compression algorithm which considers both attention and frequency.

Model	Compression		#Inf. Tok↓	#Mem. Tok↓	Acc@Ego	Acc@Movie
	Att	Freq				
MuKV			12.5K	177K	53.7	44.3
MuKV	✓		8.3K	59K	55.9	45.1
MuKV		✓	8.3K	59K	56.6	45.3
MuKV	DCP(67%)		8.3K	59K	57.3	45.6
MuKV	DCP(50%)		6.3K	89K	53.7	43.8
MuKV	DCP(75%)		3.1K	44K	54.9	45.2
ReKV [9]	—	—	12.5K	59K	51.5	42.3
ReKV	Half Frames		12.5K	29K	53.0	41.9
ReKV	Rand(50%)		12.5K	29K	46.7	34.7
ReKV	DCP(25%)		9.5K	44K	54.3	45.2
ReKV	DCP(50%)		6.3K	29K	56.1	44.9
ReKV	DCP(75%)		3.1K	15K	51.1	44.1
ReKV	DCP(90%)		1.3K	6K	50.9	46.8

Table 4. Real deployment metrics. s/Q: Average seconds needed for answering each question (online KV retrieval and QA latency). G/h: Cache size per hour of video contents.

Model	Compress Ratio	Time (s/Q)↓	KV-Cache (G/h)↓	Acc@Ego
ReKV	0	0.92	4.00	51.5
MuKV	0	0.72	3.72	53.7
MuKV	2/3	0.65	1.23	57.3
MuKV	1/2	0.67	1.85	53.7
MuKV	3/4	0.59	0.91	54.9

e.g., +5.8% and +3.3% on RVSEgo and RVSMovie respectively (Tab. 4 also shows the real deployment performances to better understand this benefit.). The bottom block shows that DCP consistently improves ReKV [9]. For instance, pruning half of the tokens using DCP (DCP(50%)) reduces ReKV’s memory size by half, and speeds up its online inference by 2×, while also increasing the accuracy by 4.6% on RVSEgo and 2.6% on RVSMovie.

To further validate such strength, we randomly drop half of the tokens in ReKV (Rand(50%)), thus maintaining the same KV cache size. The accuracy, however, decreases by 4.8%, indicating that our method benefits from informed compression rather than random token reduction. We also test a more sparse frame sampling strategy (Half Frames) to reduce memory usage. While this slightly improves performance on RVSEgo, it leads to a performance drop on RVSMovie. Further dataset analysis (see Supplementary) reveals that the average ratio of answer moments to the video length (before the question timestamp) in RVSEgo is roughly 3× that of RVSMovie. The smaller answer-moment ratio in RVSMovie makes the task akin to finding a needle in a haystack, where sparse sampling becomes detrimental. Instead, our DCP compression algorithm effectively benefits QA performance on both datasets.

Additionally, we study other compression ratios and find the optimal one to be 67% (2/3) for MuKV (We compress the KV cache to be 1/3 of ReKV since we have three gran-

Table 5. Controlled comparison with existing video KV-cache compression method (*e.g.*, InfiPot-V [19]).

LLaVA-OV-7B	#Inf. Tok↓	# Mem. Tok↓	Acc@Ego
+ReKV	12.5K	5.9K	55.8
+InfiPot-V	12.5K	6K	57.9
+MuKV (TaR+VaN)	8.3K	5.9K	58.3
+MuKV (DCP)	8.3K	5.9K	59.5

Table 6. Comparison between compressing (pruning) tokens of low and high frequency scores.

Model	Frequency		Ratio	Acc@Ego	Acc@Movie
	Low	High			
ReKV	✗	✓	50%	55.1	43.9
ReKV	✓	✗	50%	51.4	41.4

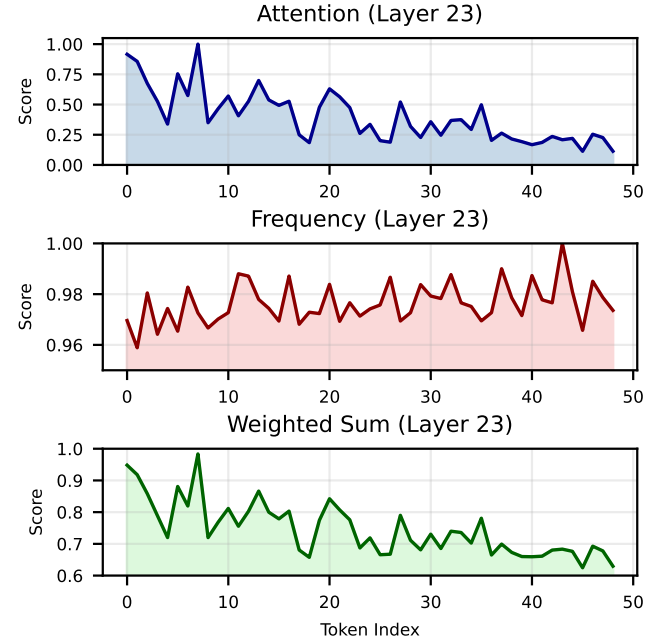


Figure 5. Distribution of tokens’ self-attention scores (top), frequency signals (middle), and their weighted sums (bottom).

ularity representations.). For ReKV, the optimal ratios are 50% on RVSEgo and 90% on RVSMovie. The larger compression ratio on RVSMovie indicates its high content redundancy, possibly due to much longer videos, *e.g.* 1 hour.

Finally, we conduct a controlled comparison with the recent video KV-cache compression approach InfiPot-V [19]. Tab. 5 shows that our method steadily surpasses both the whole InfiPot-V approach and its core compression module (TaR and VaN), suggesting DCP’s strength for video KV cache compression.

Effect of Frequency. We further examine whether retaining high- or low-frequency tokens yields better performance. In videos, high-frequency components typically correspond either to foreground details in static scenes or to rapid motion in dynamic scenes. As shown in Tab. 6, pre-

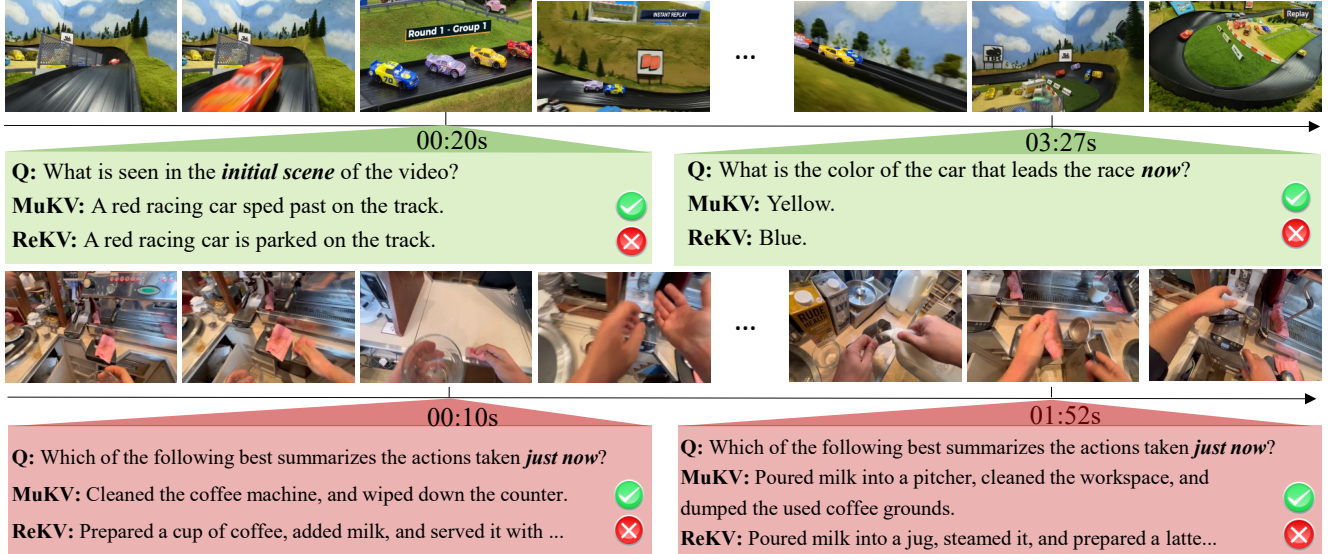


Figure 6. Prediction visualization on StreamingBench [23]. Our method (MuKV) outperforms ReKV [9]. It effectively reasons about segment-level temporal dynamics and region level details despite with extended video content inputs.

Table 7. Study of different KV retrieval methods.

Model	Para.	Hier.	Semi-Hier.	Acc@Ego	Acc@Movie
MuKV	✓			56.5	46.0
MuKV		✓		52.9	43.1
MuKV			✓	57.9	45.2

serving higher-frequency tokens consistently leads to superior results. To better illustrate the role of frequency, we visualize the tokens’ distributions of self-attention, frequency, and their combined scores from the last LLM layer in Fig. 5. Interestingly, earlier coming tokens often receive higher attentions and later tokens lower attentions regardless of samples. In contrast, the frequency distribution is more sample-specific and therefore provides an effective way to calibrate this positional bias of self-attention, ultimately enjoying the best of both and improving compression performance.

Semi-hierarchical Retrieval. Tab. 7 compares our semi-hierarchical retrieval strategy with both parallel and naive hierarchical retrieval methods. For fair comparison, we retrieve equal number of KV blocks for all three methods. For parallel retrieval, we directly return the top- k_g blocks retrieved at our first stage. For hierarchical retrieval, we retrieve only the frame- and patch-level KV blocks within the top-5 ranked segment blocks, with also the question as query in all stages. The combined KV blocks from all three granularities are loaded to LLM for answer inference. The results show that our semi-hierarchical method achieves the best results on RVSEgo while finds a balance between parallel and hierarchical retrieval on RVSMovie.

LLM Layer Investigation. While we derive the attention and frequency scores from the last LLM layer (so as

Table 8. Layer investigation on RVS-Ego.

LLaVA-OV-0.5B	Last (default)	Penultimate	Middle	Last-5 Fusion
+MuKV	57.3	54.9	53.4	56.5

for KV cache retrieval), we also investigate other alternatives. Tab. 8 shows that all other designs perform worse than the last layer representations. This is reasonable as the last-layer representations are closer to our target task that our backbone MLLMs are designed for.

5. Conclusion

We advance long streaming VideoQA by introducing MuKV, a framework that compactly stores and effectively retrieves multi-grained video KV caches for accurate and efficient answer decoding. To enable compact storage offline, we propose a dual-signal KV-cache compression module that jointly considers token self-attention importance and token frequency in the Fourier domain. For effective online retrieval, we design a semi-hierarchical retrieval mechanism that performs grain-agnostic parallel retrieval followed by cross-grain hierarchical re-ranking. Extensive experiments demonstrate that MuKV significantly improves accuracy without increasing offline memory or online inference latency, revealing substantial redundancy in existing streaming QA methods and highlighting our effectiveness for elimination towards enhancements.

Acknowledgements

This research is supported by the Ministry of Education, Singapore, under its MOE Academic Research Fund Tier 2 (MOE-T2EP20125-0037).

References

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *NeurIPS*, 35: 23716–23736, 2022. 1
- [2] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025. 6
- [3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2. 5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025. 1, 2
- [4] Dibyadip Chatterjee, Edoardo Remelli, Yale Song, Bugra Tekin, Abhay Mittal, Bharat Bhatnagar, Necati Cihan Camg z, Shreyas Hampali, Eric Sauser, Shugao Ma, et al. Memory-efficient streaming videollms for real-time procedural video understanding. *arXiv preprint arXiv:2504.13915*, 2025. 1
- [5] Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. In *European Conference on Computer Vision*, pages 19–35. Springer, 2024. 3
- [6] Xueyi Chen, Keda Tao, Kele Shao, and Huan Wang. Streamingtom: Streaming token compression for efficient video understanding. *arXiv preprint arXiv:2510.18269*, 2025. 2
- [7] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, 2023. 2
- [8] James W Cooley and John W Tukey. An algorithm for the machine calculation of complex fourier series. *Mathematics of computation*, 19(90):297–301, 1965. 4
- [9] Shangzhe Di, Zhelun Yu, Guanghao Zhang, Haoyuan Li, Hao Cheng, Bolin Li, Wanggui He, Fangxun Shu, Hao Jiang, et al. Streaming video question-answering with in-context video kv-cache retrieval. In *ICLR*, 2025. 1, 2, 3, 5, 6, 7, 8
- [10] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv e-prints*, pages arXiv–2407, 2024. 2
- [11] Yue Fan, Xiaojian Ma, Rujie Wu, Yuntao Du, Jiaqi Li, Zhi Gao, and Qing Li. Videoagent: A memory-augmented multimodal agent for video understanding. In *ECCV*, pages 75–92. Springer, 2024. 1, 2
- [12] Chaoyou Fu, Yuhao Dai, Yongdong Luo, Lei Li, Shuhuai Ren, Renrui Zhang, Zihan Wang, Chenyu Zhou, Yunhang Shen, Mengdan Zhang, et al. Video-mme: The first-ever comprehensive evaluation benchmark of multi-modal llms in video analysis. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 24108–24118, 2025. 1, 5
- [13] Yu Fu, Zefan Cai, Abedelkadir Asi, Wayne Xiong, Yue Dong, and Wen Xiao. Not all heads matter: A head-level kv cache compression method with integrated retrieval and reasoning. *ICLR*, 2025. 3
- [14] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Yakun S Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems*, 37:1270–1303, 2024. 2
- [15] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*, 2024. 2
- [16] Dongwon Jo, Jiwon Song, Yulhwa Kim, and Jae-Joon Kim. Fastkv: Kv cache compression for fast long-context processing with token-selective propagation. *arXiv preprint arXiv:2502.01068*, 2025. 2, 3
- [17] Minjoon Jung, Junbin Xiao, Byoung-Tak Zhang, and Angela Yao. On the consistency of video large language models in temporal comprehension. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 13713–13722, 2025. 2
- [18] Jushi Kai, Boyi Zeng, Yixuan Wang, Haoli Bai, Ziwei He, Bo Jiang, and Zhouhan Lin. Freqkv: Frequency domain key-value compression for efficient context window extension. *arXiv preprint arXiv:2505.00570*, 2025. 3
- [19] Minsoo Kim, Kyuhong Shim, Jungwook Choi, and Simyung Chang. Infinipot-v: Memory-constrained kv cache compression for streaming video understanding. *NeurIPS*, 2025. 1, 2, 3, 7
- [20] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, et al. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024. 2, 3, 5, 6
- [21] Kunchang Li, Yali Wang, Yinan He, Yizhuo Li, Yi Wang, Yi Liu, Zun Wang, Jilan Xu, Guo Chen, Ping Luo, et al. Mvbench: A comprehensive multi-modal video understanding benchmark. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22195–22206, 2024. 1
- [22] Bin Lin, Yang Ye, Bin Zhu, Jiaxi Cui, Munan Ning, Peng Jin, and Li Yuan. Video-llava: Learning united visual representation by alignment before projection. *arXiv preprint arXiv:2311.10122*, 2023. 1, 2
- [23] Junming Lin, Zheng Fang, Chi Chen, Zihao Wan, Fuwen Luo, Peng Li, Yang Liu, and Maosong Sun. Streamingbench: Assessing the gap for mllms to achieve streaming video understanding. *arXiv preprint arXiv:2411.03628*, 2024. 5, 8
- [24] Guangda Liu, Chengwei Li, Zhenyu Ning, Jing Lin, Yiwu Yao, Danning Ke, Minyi Guo, and Jieru Zhao. Freekv: Boosting kv cache retrieval for efficient llm inference. *arXiv preprint arXiv:2505.13109*, 2025. 3
- [25] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. Cachegen: Kv cache com-

- pression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 38–56, 2024. 2
- [26] Muhammad Maaz, Hanoona Rasheed, Salman Khan, and Fahad Shahbaz Khan. Video-chatgpt: Towards detailed video understanding via large vision and language models. *arXiv preprint arXiv:2306.05424*, 2023. 1, 2
- [27] Karttikeya Mangalam, Raiymbek Akshulakov, and Jitendra Malik. Egoschema: A diagnostic benchmark for very long-form video language understanding. *Advances in Neural Information Processing Systems*, 36:46212–46244, 2023. 5
- [28] Juhong Min, Shyamal Buch, Arsha Nagraani, Minsu Cho, and Cordelia Schmid. Morevqa: Exploring modular reasoning models for video question answering. In *CVPR*, pages 13235–13245, 2024. 2
- [29] Zhenyu Ning, Guangda Liu, Qihao Jin, Wenchao Ding, Minyi Guo, and Jieru Zhao. Livevlm: Efficient online video understanding via streaming-oriented kv cache and retrieval. *arXiv preprint arXiv:2505.15269*, 2025. 2, 3
- [30] Rui Qian, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Shuangrui Ding, Dahua Lin, and Jiaqi Wang. Streaming long video understanding with large language models. *NeurIPS*, 37: 119336–119360, 2024. 1
- [31] Hangyu Qin, Junbin Xiao, and Angela Yao. Question-answering dense video events. In *SIGIR*, pages 884–894, 2025. 1
- [32] Xiaoqian Shen, Yunyang Xiong, Changsheng Zhao, Lemeng Wu, Jun Chen, Chenchen Zhu, Zechun Liu, Fanyi Xiao, Balakrishnan Varadarajan, Florian Bordes, et al. Longvu: Spatiotemporal adaptive compression for long video-language understanding. *arXiv preprint arXiv:2410.17434*, 2024. 1, 2
- [33] Hanlin Tang, Yang Lin, Jing Lin, Qingsen Han, Shikuan Hong, Yiwu Yao, and Gongyi Wang. Razorattention: Efficient kv cache compression through retrieval heads. *ICLR*, 2025. 2
- [34] Dongwei Wang, Zijie Liu, Song Wang, Yuxin Ren, Jianing Deng, Jingtong Hu, Tianlong Chen, and Huanrui Yang. Fier: Fine-grained and efficient kv cache retrieval for long-context llm inference. *arXiv preprint arXiv:2508.08256*, 2025. 3
- [35] Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, et al. Internvl3. 5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. *arXiv preprint arXiv:2508.18265*, 2025. 1, 2
- [36] Xiaohan Wang, Yuhui Zhang, Orr Zohar, and Serena Yeung-Levy. Videoagent: Long-form video understanding with large language model as agent. In *European Conference on Computer Vision*, pages 58–76. Springer, 2024. 1, 2
- [37] Yi Wang, Xinhao Li, Ziang Yan, Yinan He, Jiashuo Yu, Xiangyu Zeng, Chenting Wang, Changlian Ma, Haian Huang, Jianfei Gao, et al. Internvideo2. 5: Empowering video mllms with long and rich context modeling. *arXiv preprint arXiv:2501.12386*, 2025. 1
- [38] Zikang Wang, Boyu Chen, Zhengrong Yue, Yi Wang, Yu Qiao, Limin Wang, and Yali Wang. Videochat-a1: Thinking with long videos by chain-of-shot reasoning. *arXiv preprint arXiv:2506.06097*, 2025. 1, 2
- [39] Ziyang Wang, Shoubin Yu, Elias Stengel-Eskin, Jaehong Yoon, Feng Cheng, Gedas Bertasius, and Mohit Bansal. Videotree: Adaptive tree-based video representation for llm reasoning on long videos. In *CVPR*, pages 3272–3283, 2025. 1, 3
- [40] Yuetian Weng, Mingfei Han, Haoyu He, Xiaojun Chang, and Bohan Zhuang. Longvlm: Efficient long video understanding via large language models. In *European Conference on Computer Vision*, pages 453–470. Springer, 2024. 1, 2
- [41] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. 2
- [42] Junbin Xiao, Xindi Shang, Angela Yao, and Tat-Seng Chua. Next-qa: Next phase of question-answering to explaining temporal actions. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9777–9786, 2021. 1
- [43] Junbin Xiao, Nanxin Huang, Hangyu Qin, Dongyang Li, Yicong Li, Fengbin Zhu, Zhulin Tao, Jianxing Yu, Liang Lin, Tat-Seng Chua, et al. Videoqa in the era of llms: An empirical study. *International Journal of Computer Vision*, 133(7): 3970–3993, 2025. 2
- [44] Dejing Xu, Zhou Zhao, Jun Xiao, Fei Wu, Hanwang Zhang, Xiangnan He, and Yueting Zhuang. Video question answering via gradually refined attention over appearance and motion. In *Proceedings of the 25th ACM international conference on Multimedia*, pages 1645–1653, 2017. 1
- [45] Lin Xu, Yilin Zhao, Daquan Zhou, Zhijie Lin, See Kiong Ng, and Jiashi Feng. Pllava: Parameter-free llava extension from images to videos for video dense captioning. *arXiv preprint arXiv:2404.16994*, 2024. 1
- [46] Ruyi Xu, Guangxuan Xiao, Yukang Chen, Liuning He, Kelly Peng, Yao Lu, and Song Han. Streamingvlm: Real-time understanding for infinite video streams. *arXiv preprint arXiv:2510.09608*, 2025. 2
- [47] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. 1, 2
- [48] Yanlai Yang, Zhuokai Zhao, Satya Narayan Shukla, Aashu Singh, Shlok Kumar Mishra, Lizhu Zhang, and Mengye Ren. Streammem: Query-agnostic kv cache memory for streaming video understanding. *arXiv preprint arXiv:2508.15717*, 2025. 3
- [49] Zhou Yu, Dejing Xu, Jun Yu, Ting Yu, Zhou Zhao, Yueting Zhuang, and Dacheng Tao. Activitynet-qa: A dataset for understanding complex web videos via question answering. In *AAAI*, pages 9127–9134, 2019. 1
- [50] Andy Zeng, Maria Attarian, Krzysztof Marcin Choromanski, Adrian Wong, Stefan Welker, Federico Tombari, Aavek Purohit, Michael S Ryoo, Vikas Sindhwani, Johnny Lee, et al. Socratic models: Composing zero-shot multimodal reasoning with language. In *ICLR*. 2
- [51] Boqiang Zhang, Kehan Li, Zesen Cheng, Zhiqiang Hu, Yuqian Yuan, Guanzheng Chen, Sicong Leng, Yuming Jiang, Hang Zhang, Xin Li, et al. Videollama 3: Frontier multi-

- modal foundation models for image and video understanding. *arXiv preprint arXiv:2501.13106*, 2025. [1](#)
- [52] Ce Zhang, Taixi Lu, Md Mohaiminul Islam, Ziyang Wang, Shoubin Yu, Mohit Bansal, and Gedas Bertasius. A simple llm framework for long-range video question-answering. In *EMNLP*, pages 21715–21737, 2024. [1](#), [2](#)
- [53] Hang Zhang, Xin Li, and Lidong Bing. Video-llama: An instruction-tuned audio-visual language model for video understanding. *arXiv preprint arXiv:2306.02858*, 2023. [1](#), [2](#)
- [54] Haoji Zhang, Yiqin Wang, Yansong Tang, Yong Liu, Jiashi Feng, Jifeng Dai, and Xiaojie Jin. Flash-vstream: Memory-based real-time understanding for long video streams. *ICCV*, 2025. [1](#), [2](#), [5](#), [6](#)
- [55] Peiyuan Zhang, Kaichen Zhang, Bo Li, Guangtao Zeng, Jingkang Yang, Yuanhan Zhang, Ziyue Wang, Haoran Tan, Chunyuan Li, and Ziwei Liu. Long context transfer from language to vision. *arXiv preprint arXiv:2406.16852*, 2024. [1](#), [2](#), [6](#)
- [56] Junjie Zhou, Yan Shu, Bo Zhao, Boya Wu, Zhengyang Liang, Shitao Xiao, Minghao Qin, Xi Yang, Yongping Xiong, Bo Zhang, et al. Mlvu: Benchmarking multi-task long video understanding. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 13691–13701, 2025. [5](#)